Event Management Dictionary System - Complete Index

Quick Links

File	Description	Use Case
README.md	SQL query patterns and documentation	Understanding how dictionaries are generated
STRUCTURE.md	Directory structure and data flow	Understanding file organization
dictionary_queries.sql	All SQL queries in one file	Direct database operations
INDEX.md	This file - complete index	Quick navigation

Individual Dictionary Type Files

Format

Each JSON file contains a single dictionary type definition following this structure:

```
{
  "dictType": "unique_identifier",
  "dictName": "Display Name (Arabic - English)",
  "status": "0",
  "remark": "SQL query pattern for generating data",
  "createBy": "system"
}
```

Files

#	File	Dict Type	Source Table	Primary Key
1	event_type.json	`event_type`	`event_types`	`typecode`
2	zone_type.json	`zone_type`	`event_zone_types`	`typecode`
3	gate_type.json	`gate_type`	`event_gate_types`	`typecode`
4	visitor_type.json	`visitor_type`	`visitor_types`	`typecode`
5	ticket_type.json	`ticket_type`	`ticket_types`	`typecode`
6	permit_type.json	`permit_type`	`permit_types`	`typecode`
7	booth_type.json	`booth_type`	`booth_types`	`typecode`
8	incident_type.json	`incident_type`	`incident_types`	`typecode`

#	File	Dict Type	Source Table	Primary Key
9	payment_method.json	payment_method	payment_methods	methodcode
10	event_status.json	event_status	event_status	typecode
11	event_location.json	event_location	event_locations	locationid
12	hr_employee.json	hr_employee	hr_employee	employeeid

Combined Files

all_event_management_dict_types.json

Purpose: Single file containing all 12 dictionary type definitions
Format: JSON array
Use Case: Bulk import of all dictionary types

Import Example:

```
# Using curl
curl -X POST http://localhost:8887/api/system/dictType/import \
  -H "Content-Type: application/json" \
  -d @all_event_management_dict_types.json

# Using C# code
var dictTypes = JsonSerializer.Deserialize<List<SysDictType>>(
    File.ReadAllText("all_event_management_dict_types.json")
);
```

SQL Queries File

dictionary_queries.sql

Contains:

1. Individual queries for each dictionary type
2. Combined UNION ALL query for all types
3. INSERT template for sys_dict_data
4. Verification queries

Sections:

- **Part 1**: Individual SELECT queries (12 queries)
- **Part 2**: Combined UNION ALL query
- **Part 3**: INSERT INTO sys_dict_data template
- **Part 4**: Verification and reporting queries

Usage:

```
# Execute individual query
psql -d MahamDB -f dictionary_queries.sql

# Execute specific section
psql -d MahamDB -c "$(sed -n '/-- 1. Event Types/,/ORDER BY/p'
dictionary_queries.sql)"
```

SQL Query Patterns

Standard Pattern (Most Types)

```
SELECT
    typecode AS dictValue,
    CONCAT(typecode, ' - ', typename, ' - ', typenamear) AS dictLabel,
    remark
FROM [table_name]
```

Used By: event_type, zone_type, gate_type, visitor_type, ticket_type, permit_type, booth_type, incident_type, event_status

Payment Method Pattern

```
SELECT
    methodcode AS dictValue,
    CONCAT(methodcode, ' - ', methodname, ' - ', methodnamear) AS dictLabel,
    remark
FROM payment_methods
```

Event Location Pattern

```
SELECT
    locationid AS dictValue,
    CONCAT(locationid, ' - ', locationname, ' - ', city) AS dictLabel,
    address AS remark
FROM event_locations
```

HR Employee Pattern (Original)

```
SELECT
    employeeid AS dictValue,
    CONCAT(firstname, ' ', lastname, ' ', namear) AS dictLabel,
```

```
    remark
FROM hr_employee
```

Implementation Reference

C# Code Location

File: `ZR.ServiceCore/Services/SeedDataService.cs`

Key Methods:

Method	Lines	Description
`GenerateEventManagementDictTypes()`	1346-1449	Creates all dict type definitions
`GenerateDictDataFromEventTypes()`	1451-1482	Generates event type dict data
`GenerateDictDataFromZoneTypes()`	1484-1504	Generates zone type dict data
`GenerateDictDataFromGateTypes()`	1506-1526	Generates gate type dict data
`GenerateDictDataFromVisitorTypes()`	1528-1548	Generates visitor type dict data
`GenerateDictDataFromTicketTypes()`	1550-1570	Generates ticket type dict data
`GenerateDictDataFromPermitTypes()`	1572-1592	Generates permit type dict data
`GenerateDictDataFromBoothTypes()`	1594-1614	Generates booth type dict data
`GenerateDictDataFromIncidentTypes()`	1616-1636	Generates incident type dict data
`GenerateDictDataFromPaymentMethods()`	1638-1658	Generates payment method dict data
`GenerateDictDataFromEventStatus()`	1660-1680	Generates event status dict data
`GenerateDictDataFromEventLocations()`	1682-1702	Generates event location dict data
`GenerateDictDataFromHrEmployees()`	1704-1725	Generates HR employee dict data
`InitEventManagementDictTypes()`	1727-1731	Inserts/updates dict types in DB
`InitGeneratedDictData()`	1737-1748	Inserts/updates dict data in DB
`GenerateAndSaveAllDictionaries()`	1754-1845	Master orchestration method

Usage Scenarios

Scenario 1: Initial Setup

Goal: First-time setup of dictionary system

Steps:

1. Import source data to Event Management tables
2. Run seed data initialization:

```
POST /api/system/seed/initialize
```

3. System automatically:
    - Generates all dictionary types
    - Generates all dictionary data
    - Saves JSON files
    - Inserts into database

Scenario 2: Add New Event Type

Goal: Add a new event type and update dictionaries

Steps:

1. Insert into source table:

```
INSERT INTO event_types (typecode, typename, typenamear, remark)
VALUES ('CONF', 'Conference', 'مؤتمر', 'Conference event type');
```

2. Re-run seed initialization or call:

```
var results =
seedDataService.GenerateAndSaveAllDictionaries(webRootPath);
```

3. New entry automatically added to:
    - `sys_dict_data` table
    - JSON files

Scenario 3: Manual Import

Goal: Import dictionary types without running full seed initialization

Using API:

```
# Import dictionary types
curl -X POST http://localhost:8887/api/system/dictType/import \
  -H "Content-Type: application/json" \
  -d @all_event_management_dict_types.json
```

Using SQL:

```sql
-- Load from JSON file (PostgreSQL with json extension)
INSERT INTO sys_dict_type (dict_type, dict_name, status, remark, create_by)
SELECT
    j->>'dictType',
    j->>'dictName',
    j->>'status',
    j->>'remark',
    j->>'createBy'
FROM json_array_elements(
    pg_read_file('all_event_management_dict_types.json')::json
) AS j;
```

Scenario 4: Query Dictionary Data

Goal: Get dictionary data for dropdowns

C# Code:

```csharp
// Get all event types
var eventTypes = await _dictDataService
    .GetDictDataByType("event_type");

// Get active employees
var employees = await _dictDataService
    .GetDictDataByType("hr_employee")
    .Where(d => d.Status == "0");

// Get locations for specific city
var locations = await _db.Queryable<SysDictData>()
    .Where(d => d.DictType == "event_location"
            && d.DictLabel.Contains("Riyadh"))
    .ToListAsync();
```

SQL:

```sql
-- Get all event types for dropdown
SELECT dict_value, dict_label
FROM sys_dict_data
WHERE dict_type = 'event_type' AND status = '0'
ORDER BY dict_sort;

-- Get all dictionaries for Event Management
SELECT dict_type, COUNT(*) as count
FROM sys_dict_data
WHERE dict_type IN (
    'event_type', 'zone_type', 'gate_type',
    'visitor_type', 'ticket_type', 'permit_type',
```

```
    'booth_type', 'incident_type', 'payment_method',
    'event_status', 'event_location', 'hr_employee'
)
GROUP BY dict_type;
```

Maintenance Tasks

Weekly

- Review generated JSON files for accuracy
- Verify dictionary data matches source tables

Monthly

- Clean up unused dictionary entries
- Update documentation if new types added

As Needed

- Add new dictionary types when new entities are created
- Update SQL patterns if source table schemas change
- Regenerate all dictionaries after bulk data imports

Troubleshooting

Issue: Dictionary data not generated

Check:

1. Source tables have data
2. Column names match expected pattern
3. Seed initialization completed without errors
4. File permissions allow JSON writes

Solution:

```
// Check logs
var results = seedDataService.GenerateAndSaveAllDictionaries(webRootPath);
foreach (var msg in results)
{
    Console.WriteLine(msg);
}
```

Issue: Duplicate dictionary entries

Cause: Multiple seed initializations without cleanup

Solution:

```sql
-- Remove duplicates, keep newest
DELETE FROM sys_dict_data a
USING sys_dict_data b
WHERE a.dict_code < b.dict_code
  AND a.dict_type = b.dict_type
  AND a.dict_value = b.dict_value;
```

Issue: Missing dictionary types

Check:

```sql
-- Verify all types exist
SELECT dict_type, dict_name
FROM sys_dict_type
WHERE dict_type IN (
    'event_type', 'zone_type', 'gate_type',
    'visitor_type', 'ticket_type', 'permit_type',
    'booth_type', 'incident_type', 'payment_method',
    'event_status', 'event_location', 'hr_employee'
);
```

Solution: Re-import from `all_event_management_dict_types.json`

Related Files

Source Data

- `../SeedData/EventManagement/*.json` - Source data files
- `../SeedData/sys_dict_type/EventManagement/*.json` - Generated dictionary data

Code

- `ZR.ServiceCore/Services/SeedDataService.cs` - Implementation
- `ZR.Model/System/Model/SysDictType.cs` - Dict type model
- `ZR.Model/System/Model/SysDictData.cs` - Dict data model

Documentation

- `README.md` - SQL patterns and usage
- `STRUCTURE.md` - Directory structure and data flow
- `dictionary_queries.sql` - All SQL queries
- `INDEX.md` - This file

Summary Statistics

Metric	Count

Metric	Count
Total Dictionary Types	12
Individual JSON Files	12
Combined JSON Files	1
Documentation Files	4
SQL Files	1
C# Methods	15
Total Files	16
Total Lines of Code (C#)	~500
Total SQL Queries	12+

Version History

Version	Date	Changes
1.0	2025-10-20	Initial creation with all 12 dictionary types

Last Updated: October 20, 2025
Maintained By: System Architecture Team
Contact: For questions or issues, please refer to the main README.md or create an issue in the project repository.